Sri Lanka Institute of Information Technology

Lab Submission
<05>

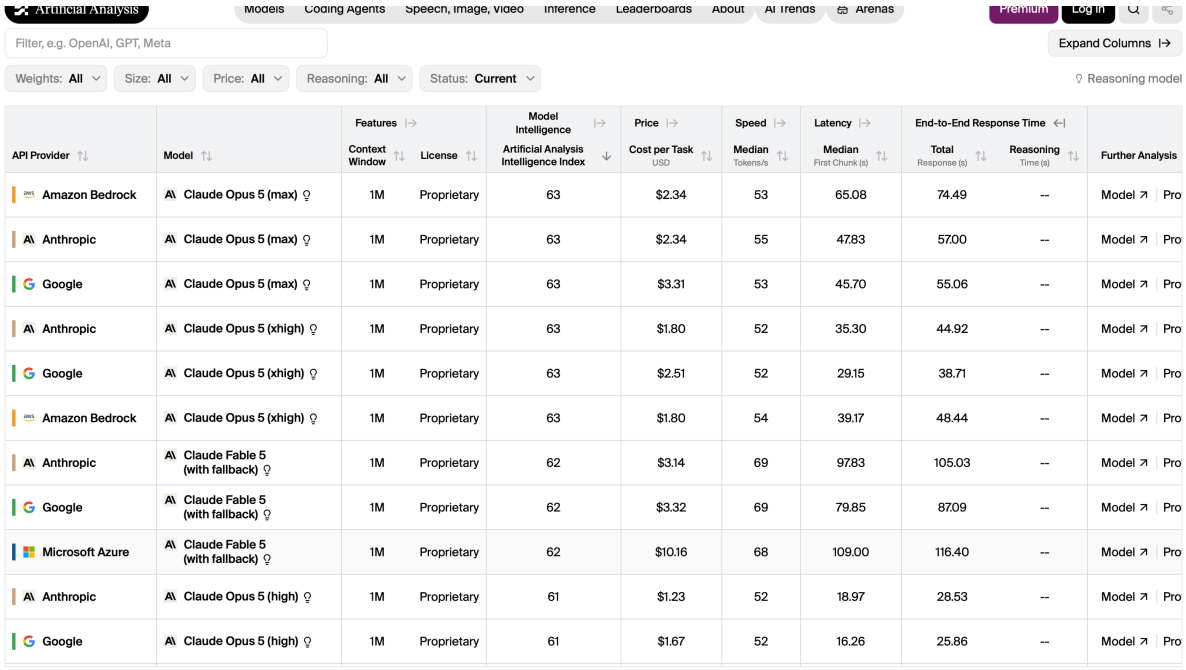
<IT25100429>
<Hiruka Liyanage>

AI/ML | IT2011

B.Sc. (Hons) in Information Technology

Exercise 02

I am using Claude Fable 5



Artificial Analysis | Models | Coding Agents | Speech, Image, Video | Inference | Leaderboards | About | AI Trends | Arenas | Premium | Log In

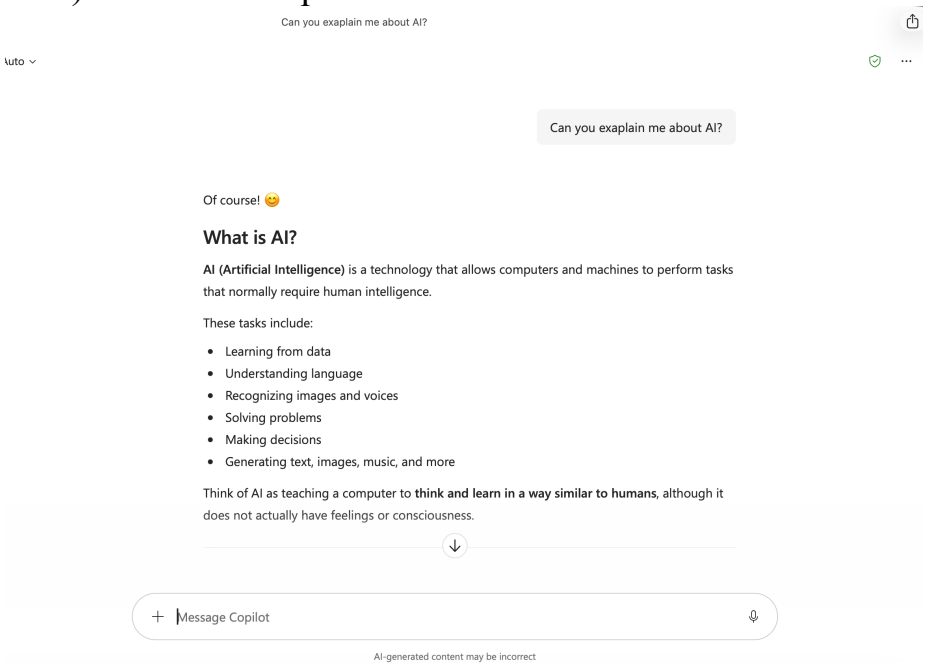
Filter, e.g. OpenAI, GPT, Meta | Expand Columns

Weights: All | Size: All | Price: All | Reasoning: All | Status: Current | Reasoning model

API Provider	Model	Features: Context Window	Features: License	Model Intelligence: Artificial Analysis Intelligence Index	Price: Cost per Task USD	Speed: Median Tokens/s	Latency: Median First Chunk (s)	End-to-End Response Time: Total Response (s)	End-to-End Response Time: Reasoning Time (s)	Further Analysis
Amazon Bedrock	Claude Opus 5 (max)	1M	Proprietary	63	$2.34	53	65.08	74.49	--	Model ↗ | Pro
Anthropic	Claude Opus 5 (max)	1M	Proprietary	63	$2.34	55	47.83	57.00	--	Model ↗ | Pro
Google	Claude Opus 5 (max)	1M	Proprietary	63	$3.31	53	45.70	55.06	--	Model ↗ | Pro
Anthropic	Claude Opus 5 (xhigh)	1M	Proprietary	63	$1.80	52	35.30	44.92	--	Model ↗ | Pro
Google	Claude Opus 5 (xhigh)	1M	Proprietary	63	$2.51	52	29.15	38.71	--	Model ↗ | Pro
Amazon Bedrock	Claude Opus 5 (xhigh)	1M	Proprietary	63	$1.80	54	39.17	48.44	--	Model ↗ | Pro
Anthropic	Claude Fable 5 (with fallback)	1M	Proprietary	62	$3.14	69	97.83	105.03	--	Model ↗ | Pro
Google	Claude Fable 5 (with fallback)	1M	Proprietary	62	$3.32	69	79.85	87.09	--	Model ↗ | Pro
Microsoft Azure	Claude Fable 5 (with fallback)	1M	Proprietary	62	$10.16	68	109.00	116.40	--	Model ↗ | Pro
Anthropic	Claude Opus 5 (high)	1M	Proprietary	61	$1.23	52	18.97	28.53	--	Model ↗ | Pro
Google	Claude Opus 5 (high)	1M	Proprietary	61	$1.67	52	16.26	25.86	--	Model ↗ | Pro

Two New Models:- Qs- Can you explain me about AI?

1) Microsoft Copilot



What is AI?

AI (Artificial Intelligence) is a technology that allows computers and machines to perform tasks that normally require human intelligence.

These tasks include:

- Learning from data
- Understanding language
- Recognizing images and voices
- Solving problems
- Making decisions
- Generating text, images, music, and more

Think of AI as teaching a computer to **think and learn in a way similar to humans**, although it does not actually have feelings or consciousness.

Simple Example

When you use:

- **ChatGPT or Microsoft Copilot** → AI understands your questions and generates answers.
- **Google Maps** → AI finds the best route.
- **Netflix or YouTube** → AI recommends videos you might like.
- **Face Unlock on a phone** → AI recognizes your face.

How AI Works

AI learns from large amounts of **data**.

For example:

1. Thousands of pictures of cats are given to an AI system.
2. The AI analyzes patterns such as ears, eyes, and shape.
3. After learning, it can recognize a new cat picture it has never seen before.

This process is called **Machine Learning (ML)**.

Types of AI

1. Narrow AI (Weak AI)

Designed for specific tasks.

Examples:

- Siri
- Alexa
- Google Translate
- ChatGPT

This is the type of AI we use today.

2. General AI (Strong AI)

A future concept where AI can perform any intellectual task a human can do.

This type does **not yet exist**.

Important AI Fields

Machine Learning (ML)

AI learns from data and improves over time.

Deep Learning

A more advanced form of machine learning using artificial neural networks inspired by the human brain.

Natural Language Processing (NLP)
Allows computers to understand and generate human language.
Examples:

- Chatbots
- Language translation
- Voice assistants

Computer Vision
Allows computers to understand images and videos.
Examples:

- Facial recognition
- Self-driving cars
- Medical image analysis

Advantages of AI
✅ Works quickly
✅ Can analyze large amounts of data
✅ Available 24/7
✅ Reduces repetitive work
✅ Helps in healthcare, education, business, and research

Challenges of AI
⚠️ May replace some jobs
⚠️ Can make mistakes if trained on poor data
⚠️ Raises privacy concerns
⚠️ Can be misused
⚠️ Requires responsible and ethical use

AI in Everyday Life
You probably use AI every day without noticing:

- Social media feeds
- Online shopping recommendations
- Email spam filters
- Navigation apps
- Virtual assistants
- Online customer support chatbots

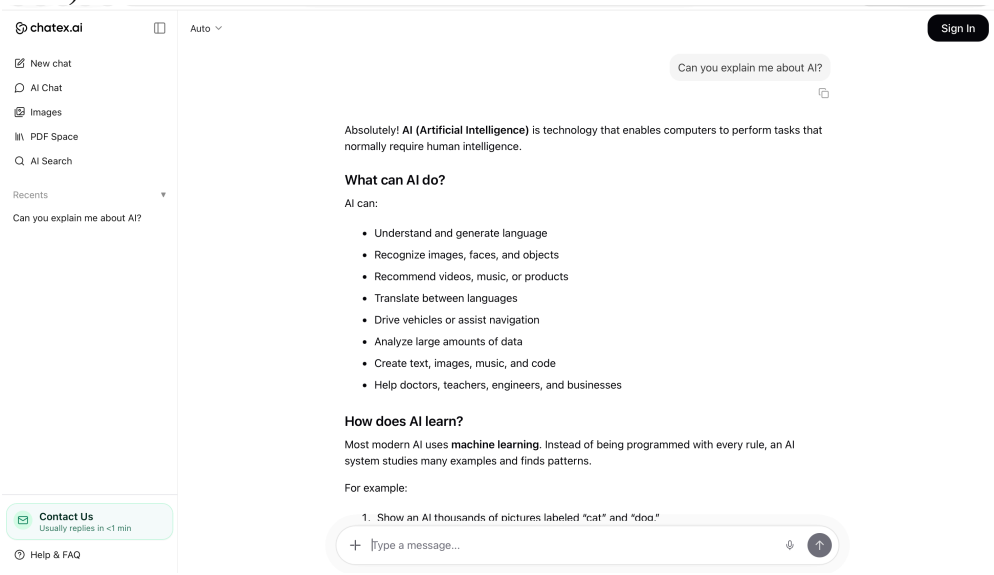
In One Sentence
Artificial Intelligence (AI) is the ability of computers to learn, reason, and perform tasks that normally require human intelligence.
Since you're an IT student, a good next step would be to learn:

1. Python programming
2. Machine Learning basics
3. Data Science

4. Deep Learning
5. Generative AI (like ChatGPT and Copilot)

These are some of the most in-demand AI skills today. 🚀

2) Chatex.ai



Absolutely! AI (Artificial Intelligence) is technology that enables computers to perform tasks that normally require human intelligence.

What can AI do?

AI can:

- Understand and generate language
- Recognize images, faces, and objects
- Recommend videos, music, or products
- Translate between languages
- Drive vehicles or assist navigation
- Analyze large amounts of data
- Create text, images, music, and code
- Help doctors, teachers, engineers, and businesses

How does AI learn?

Most modern AI uses machine learning. Instead of being programmed with every rule, an AI system studies many examples and finds patterns.
For example:

1. Show an AI thousands of pictures labeled "cat" and "dog."
2. The AI identifies patterns—ears, shapes, fur, faces, and so on.
3. When shown a new picture, it predicts whether it is a cat or a dog.

A more advanced form is deep learning, which uses large networks of connected mathematical units called neural networks.

What is generative AI?

Generative AI creates new content. For example:

- ChatGPT generates answers and explanations.
- Image generators create pictures from descriptions.
- Music tools compose songs.
- Coding assistants write or improve programs.

Large language models, such as ChatGPT, learn patterns from huge amounts of text. They generate responses by predicting what words are likely to come next. They can be very useful, but they do not think and understand exactly like a human.

What are AI's limitations?

AI can:

- Make factual mistakes
- Reflect bias in its training data
- Misunderstand questions
- Sound confident even when it is wrong
- Struggle with common sense or unusual situations
- Need human supervision for important decisions

So, AI is best viewed as a powerful assistant—not an all-knowing machine overlord. 🙂

In simple terms: AI is software that learns patterns from information and uses those patterns to make predictions, decisions, or create things.

Exercise 03

1) **Chat.deepseek.com**

How University Students Can Use Generative AI to Learn Probability & Statistics, Database Management Systems, and Software Engineering

1. Introduction

Generative Artificial Intelligence (GenAI)—including tools like ChatGPT, Google Gemini, GitHub Copilot, and Cursor—is rapidly transforming higher education. For university students in computer science and related fields, these tools offer unprecedented opportunities for personalized, interactive, and efficient learning. However, using GenAI effectively requires more than simply asking for answers; it demands a strategic approach that balances efficiency with genuine understanding.

This report provides a comprehensive, evidence-based guide on how students can leverage GenAI to master three core subjects: **Probability and Statistics**, **Database Management Systems (DBMS)**, and **Software Engineering**. Each section presents specific use cases, practical strategies, and important limitations, drawing on recent research and classroom experiences.

---

2. Probability and Statistics

Probability and statistics are foundational subjects that often challenge students with abstract concepts, complex derivations, and nuanced interpretations. GenAI can serve as a powerful learning companion across multiple dimensions.

2.1 Personalized Tutoring and Concept Explanation

One of the most immediate benefits of GenAI is its ability to act as a 24/7 personalized tutor. Students struggling with abstract concepts can ask the AI to explain ideas in multiple ways—through analogies, real-world examples, or step-by-step derivations.

Research has demonstrated the effectiveness of this approach. A study evaluating ChatGPT on 23 probability problems from first-year computer engineering exams found that **ChatGPT outperformed the average student in writing, organization, and logical reasoning**, performing consistently well in both Spanish and English. Another controlled experiment with five university students showed that after using ChatGPT support, **mean test scores rose from 39.00% to 70.50%**, with the most notable improvement in advanced-level tasks (success rate increasing from 17.50% to 47.50%).

Practical strategies:

- **Multi-perspective explanations**: Ask the AI to explain the same concept from different angles. For example: "*Explain the Central Limit Theorem using: (a) a mathematical derivation, (b) a real-world business example, and (c) an intuitive analogy.*"
- **Step-by-step derivation**: Request detailed derivations: "*Walk me through the proof of Bayes' Theorem step by step, explaining the rationale behind each step.*"
- **Concept mapping**: Ask the AI to show how different concepts relate: "*How are the normal distribution, t-distribution, and chi-squared distribution connected? Create a concept map.*"

2.2 Custom Exercise Generation and Self-Assessment

GenAI can generate unlimited practice problems tailored to a student's current level and specific areas of weakness. This is particularly valuable when textbook problems are limited or when students need extra practice on specific topics.

An experiment at the University of Debrecen asked ChatGPT to generate combinatorics, classical probability, and Bayes' Theorem exercises with solutions for each student. Students then solved their assigned problems and cross-checked against the AI-generated solutions, learning both the content and the tool's limitations. This approach ensured personalized practice while developing critical evaluation skills.

Practical strategies:

- **Targeted practice**: "*Generate 5 medium-difficulty problems on binomial distribution with detailed solutions.*"
- **Error-based reinforcement**: After making a mistake, ask: "*I got this problem wrong. Generate 3 similar problems with different contexts so I can practice the same concept.*"
- **Progressive difficulty**: "*Start with 3 basic problems on conditional probability, then 3 intermediate, then 2 challenging ones.*"

- **Self-testing**: Generate a full practice test, complete it without AI assistance, then use the AI to check answers and explain mistakes.

2.3 Visualization and Simulation

Visualizing statistical distributions and simulating random processes can dramatically improve conceptual understanding. GenAI can help students create interactive visualizations without requiring deep programming expertise.

A professor at George Washington University demonstrated "vibe coding"—using natural language prompts to generate code—with Google Gemini to create a web application for visualizing probability distributions. The application included interactive pages for the **t-distribution** (with options for two-tailed, one-tailed lower, and one-tailed upper tests), **chi-squared distribution**, and **F-ratio distribution**. Students could adjust parameters with sliders and see probability density functions, critical values, and rejection regions update in real time.

Practical strategies:

- **Generate visualization code**: "*Write Python code using matplotlib to plot a normal distribution PDF and shade the area representing P(X > 1.96).*"
- **Create interactive tools**: "*Generate an HTML/JavaScript page with a slider for degrees of freedom that updates a t-distribution plot in real time.*"
- **Run simulations**: "*Write a Python script that simulates rolling two dice 10,000 times and plots the distribution of sums to demonstrate the Central Limit Theorem.*"
- **Explore concepts dynamically**: Use AI to generate code that lets you manipulate parameters and observe changes immediately, making abstract concepts tangible.

2.4 Important Limitations

Despite its strengths, GenAI has notable weaknesses in probability and statistics. Research shows that **ChatGPT has difficulties with basic numerical operations**, although returning the solution via an R script helps it overcome these limitations. Students should therefore:

- **Cross-validate numerical answers** using calculators or statistical software (R, Python, Excel).
- **Focus on reasoning, not just answers**—the AI's step-by-step logic is often more valuable than the final numeric result.

- **Be aware that ChatGPT tends to favor empirical-concrete perspectives** over formal-abstract ones in its explanations, which may not always align with course expectations.

---

3. Database Management Systems

Database Management Systems (DBMS) courses require mastering conceptual design, SQL programming, normalization, transactions, and real-world problem-solving. GenAI can support learning across all these areas.

3.1 SQL Query Learning and Practice

SQL is a core skill in DBMS courses, and GenAI can serve as an intelligent SQL tutor that provides immediate feedback, explains query logic, and generates personalized exercises.

A teaching case integrating ChatGPT into a database management course through a capstone project found that **students engaged with practical tasks such as designing relational databases, generating SQL queries, and maintaining databases in response to dynamic business requirements**. ChatGPT played a central role in **providing immediate feedback, helping students refine their understanding, and allowing them to explore more advanced database operations at their own pace**. Quantitative evaluation revealed **significant performance improvements in graduate cohorts**.

Another study on GenAI-integrated SQL practical tests analyzed student performance and problem-solving strategies through their interactions with GenAI during SQL programming tests. The findings inform how to design effective AI-integrated assessments.

Practical strategies:

- **Query practice with feedback**: "*Here's a database schema with tables Students, Courses, and Enrollments. Write a query to find students enrolled in more than 3 courses. Then explain your query.*"
- **Error diagnosis**: Input a malfunctioning query: "*This query returns an error. What's wrong and how do I fix it?*"
- **Query optimization**: "*This query runs slowly on a large database. How can I optimize it? Explain your reasoning.*"

- **Step-by-step guidance**: For complex queries, ask the AI to guide you: "*Don't give me the full query yet. First, help me write the subquery, then we'll build the outer query together.*"
- **Natural language to SQL**: Practice translating business questions into SQL: "*Convert this request into SQL: 'Find the names of all customers who purchased a product in the last 30 days.'*"

3.2 Database Design and Normalization

Conceptual modeling and normalization are often challenging for students. GenAI can help by providing realistic scenarios, guiding the design process, and reviewing student work.

Research on rethinking class exercises for teaching database management in the GenAI era emphasizes **fostering students' critical thinking, interpretation skills, and conceptual understanding of topics such as data modeling and SQL query specification**. Exercises are designed to embrace GenAI as a learning tool rather than treating it as a threat.

Practical strategies:

- **Scenario-based design**: "*I need to design a database for a library management system. Walk me through the process from requirements gathering to ER diagram to normalized tables.*"
- **Normalization practice**: "*Here's an unnormalized table for an order management system. Guide me through normalizing it to 3NF, explaining each step.*"
- **Design review**: "*Here's my database schema for an e-commerce platform. Review it for potential issues—redundancy, anomalies, missing relationships—and suggest improvements.*"
- **Compare alternatives**: "*Show me two different ways to model this many-to-many relationship and explain the trade-offs.*"

3.3 Performance and Effectiveness Data

Empirical evidence supports the effectiveness of AI-assisted DBMS learning. One study found that in an experimental class, **SQL homework correctness improved by 82%, and complex query completion time decreased by 53%**. However, the same study also found that **64% of students developed debugging dependency tendencies**. This highlights a crucial tension: GenAI can significantly reduce cognitive load and improve efficiency, but educators and students must **balance efficiency with genuine skill development**.

Practical strategies to avoid dependency:

- **Attempt first, then consult AI**: Always try to write queries independently before asking for help.
- **Require explanations**: When the AI provides a query, ask it to explain each clause and why it chose that approach.
- **Schedule AI-free practice**: Set aside time for practice without any AI assistance to ensure foundational skills are solid.
- **Use AI for review, not creation**: After completing a query independently, use AI to review and suggest optimizations.

---

4. Software Engineering

Software Engineering courses emphasize the full development lifecycle—requirements analysis, design, coding, testing, documentation, and maintenance. GenAI is transforming this field profoundly, and students must learn to use these tools effectively while developing essential engineering skills.

4.1 AI-Assisted Coding and Pair Programming

AI coding assistants like GitHub Copilot, Cursor, and ChatGPT can accelerate coding tasks, reduce syntax errors, and serve as example-driven learning tools. They function like an always-available pair programmer.

A 10-week case study with 25 engineering students developing a CRM system found that **AI-assisted programming tools saved 30-40% of time in prototyping and debugging**. Student satisfaction was extremely high—**96% expressed satisfaction with the AI-assisted learning experience**.

Another study on the effects of GitHub Copilot on undergraduate students found that **students completed tasks 35% faster and made 50% more solution progress** when using Copilot. They spent **11% less time manually writing code**and **12% less time conducting web searches**.

Brown University created a course called "Agentic Studio" to help students learn **what coding agents can and can't do, where they add value, where they fall apart, and what role human software engineers have in working with AI agents**. Students worked on real projects—from a graduation checker to a messaging app—while keeping journals of their AI experiences.

Practical strategies:

- **Use IDE-integrated tools**: Install GitHub Copilot or Cursor in your development environment for real-time code suggestions.
- **Generate boilerplate efficiently**: Use AI to generate repetitive code (e.g., CRUD operations, getters/setters) so you can focus on complex logic.
- **Request code explanations**: When the AI generates code you don't fully understand: "*Explain this code line by line. Why did you use this approach?*"
- **Refactoring assistance**: "*Refactor this function to be more readable and maintainable. Explain your changes.*"

4.2 Debugging and Testing

Debugging is a critical skill that is often under-taught. GenAI can provide structured debugging practice and help students develop systematic testing habits.

Researchers developed **BugSpotter**, a tool that uses LLMs to generate buggy code from problem descriptions. Students practice debugging by designing failing test cases that reveal the defects. This approach provides scalable, personalized debugging practice that would be impossible for instructors to create manually at scale.

Practical strategies:

- **Deliberate debugging practice**: "*Generate a Python function that contains 3 logical errors. I'll try to find and fix them, then you can verify my work.*"
- **Error analysis**: When encountering an error: "*Here's my code and the error message. Explain what's causing this and how to fix it, but let me write the fix myself.*"
- **Test case design**: "*For this function, help me design 5 boundary test cases that would catch common edge cases.*"
- **Code review simulation**: "*Act as a senior developer. Review this code for bugs, security issues, and maintainability problems.*"

4.3 Project-Based Learning and the Full Development Lifecycle

Software engineering is best learned through projects, and GenAI can support every phase of the development lifecycle.

A project-based software engineering course at the University of British Columbia found that **students utilized AI tools in all stages of project development—to generate and refine artifacts and to learn unfamiliar concepts**. AI tools **simplified repetitive development tasks and helped students quickly ramp up when working with unfamiliar frameworks and programming languages**. Importantly, the researchers observed that **skills such as requirements**

engineering, design, testing, and code review are becoming more relevant (and easier to teach) than ever in the AI era.

Practical strategies by phase:

- **Requirements analysis**: "*Here's a project description. Help me identify functional and non-functional requirements, potential risks, and stakeholder concerns.*"
- **System design**: "*Based on these requirements, suggest an architecture. Compare microservices vs. monolithic approaches for this use case.*"
- **Implementation**: Use AI for code generation, but always review and understand every line.
- **Testing**: "*Generate unit tests for this class using pytest. Include edge cases.*"
- **Documentation**: "*Generate API documentation from these code comments*" or "*Write a README for this project.*"
- **Code review**: "*Review this pull request and suggest improvements.*"

4.4 Critical Limitations and Risks

While GenAI offers tremendous benefits in software engineering, it also introduces significant risks that students must understand and mitigate.

Comprehension-Performance Decoupling: A study on code comprehension with GitHub Copilot found that **despite significant performance improvements, participants showed no overall comprehension improvement**, revealing a "comprehension-performance decoupling". However, **engaging in verification loops—actively reviewing generated code—strongly predicted comprehension**($p < 0.001$), with high-comprehension participants **verifying code 4.7 times more frequently** than low-comprehension participants.

Over-Reliance Risks: Students may accept AI-generated solutions without fully understanding the logic behind them. A study found that while GenAI tools can improve project scores, **they are less effective—and may even exacerbate disparities—in fostering long-term deeper learning and developing transformative knowledge and critical thinking**.

Code Quality Concerns: Maintaining code quality requires systematic human oversight. AI-generated code can contain subtle bugs, security vulnerabilities, or architectural flaws.

Practical strategies to mitigate risks:

- **Verification loops**: Always review AI-generated code critically before using it. Ask: "*Does this actually solve the problem? Is there a better approach? Are there edge cases it misses?*"

- **Understand before using**: Never use code you don't fully understand. If you can't explain it, you haven't learned it.
- **Balance AI and manual coding**: Use AI for efficiency, but regularly practice coding without assistance to build foundational skills.
- **Treat AI as a collaborator, not a replacement**: AI is a tool to augment your capabilities, not a substitute for your judgment and expertise.

---

5. Cross-Cutting Principles for Effective GenAI Use

Regardless of the subject, several principles apply to using GenAI effectively as a student:

5.1 Prompt Engineering

The quality of AI output depends heavily on the quality of the input. Specific, well-structured prompts yield much better results than vague requests.

Tips:

- Be specific about what you want (format, length, difficulty level, perspective).
- Provide context (your current level, what you already understand, what's confusing).
- Request explanations, not just answers.
- Ask the AI to show its work (step-by-step reasoning).

5.2 Critical Evaluation

Never take AI outputs at face value. GenAI can produce confident-sounding but incorrect information ("hallucinations").

Tips:

- Cross-check important information with course materials, textbooks, or trusted sources.
- Test AI-generated code thoroughly before relying on it.
- Compare AI explanations with what you learn in lectures and readings.
- Develop a healthy skepticism—treat AI as a helpful assistant, not an infallible authority.

5.3 Balancing Efficiency and Learning

The greatest risk of GenAI in education is that students use it to bypass learning rather than enhance it.

Tips:

- Use the **"attempt first" rule**: Always try to solve problems independently before consulting AI.
- When AI helps you, **reflect on what you learned**: What was the key insight? What would you do differently next time?
- Schedule **AI-free study sessions** to ensure your skills don't atrophy.
- Use AI to **deepen understanding**, not to avoid understanding.

5.4 Academic Integrity

Institutions have varying policies on GenAI use. Students must understand and comply with their course's policies regarding AI assistance.

Tips:

- Clarify with instructors what constitutes acceptable use.
- When in doubt, disclose your use of AI.
- Use AI as a learning tool, not as a means to complete assignments dishonestly.
- Remember that the goal is to **learn**, not just to get the right answer.

---

6. Conclusion

Generative AI offers university students powerful new ways to learn probability and statistics, database management systems, and software engineering. In probability and statistics, AI can serve as a personalized tutor, generate custom practice problems, and help create interactive visualizations. In database management, AI can provide instant SQL feedback, guide database design, and offer personalized practice. In software engineering, AI coding assistants can accelerate development, support debugging practice, and assist across the full project lifecycle.

However, the effectiveness of GenAI as a learning tool depends entirely on how students use it. Those who treat AI as a **collaborative learning partner**—using it to generate explanations they then critically evaluate, to create practice problems they then solve independently, and to accelerate routine tasks so they can focus on

deeper understanding—will see significant benefits. Those who use AI as a **substitute for learning** will find their skills stagnant.

The research is clear: GenAI can improve performance metrics dramatically, but genuine learning requires active engagement, critical thinking, and verification. The most successful students will be those who learn to **collaborate with AI** while maintaining their own judgment, creativity, and foundational knowledge.

As one software engineering educator put it: "In the new AI-empowered reality, skills such as requirements engineering, design, testing, and code review are becoming more relevant than ever". The same is true across all subjects—GenAI changes what students need to learn and how they learn it, but it does not eliminate the need for deep understanding, critical thinking, and genuine skill development

2) Chat.qwen.ai

Comprehensive Report: Leveraging Generative AI for Academic Excellence in STEM

Prepared for: University Students (Computer Science, Data Science, and Related Fields)
Subject Focus: Probability & Statistics, Database Management Systems (DBMS), and Software Engineering
Date: October 2023

---

1. Executive Summary
Generative AI (GenAI) has transitioned from a novelty to a fundamental tool in higher education. For university students, the goal is not to use AI to bypass learning, but to use it as a **personalized tutor, a sounding board, and an accelerator for comprehension**. This report provides a detailed, subject-specific guide on how to effectively and ethically use GenAI to master Probability & Statistics, DBMS, and Software Engineering.

---

2. General Principles of AI-Assisted Learning
Before diving into specific subjects, students must adopt the right mindset:
* **The Co-Pilot Mindset:** AI is your co-pilot; you are the pilot. You must verify its outputs, guide its direction, and do the actual cognitive heavy lifting.
* **Active vs. Passive Learning:** Do not just ask AI for answers. Ask it to guide you to the answer. Use the **Socratic Method** (e.g., *"Don't give me the answer, just give me a hint on the first step"*).
* **Iterative Prompting:** Treat AI interactions as a conversation. Refine your prompts based on its initial output.

---

3. Subject 1: Probability and Statistics

Core Challenges: Abstract mathematical concepts, complex formulas, and bridging the gap between theoretical math and real-world data application.

How to Use GenAI
A. Conceptual Simplification and Analogies
Statistics is full of counter-intuitive concepts. Use AI to translate dense textbook definitions into intuitive analogies.
* **Prompt Example:** *"Explain the difference between Bayesian and Frequentist probability using a real-world analogy involving a medical test for a rare disease. Keep the tone accessible for a sophomore undergraduate."*

B. Step-by-Step Problem Solving (The "Tutor" Mode)
Never ask the AI to just solve a problem. Ask it to walk you through the logic.
* **Prompt Example:** *"I need to calculate the expected value and variance for a binomial distribution where n=50 and p=0.2. Do not give me the final answer. Instead, outline the formulas I need to use, explain what each variable represents, and ask me to calculate the first step."*

C. Code Translation (Math to Python/R)
Modern statistics requires programming. Use AI to bridge the gap between mathematical notation and code.
* **Prompt Example:** *"Here is the mathematical formula for a two-sample t-test: [insert formula]. Show me how to implement this from scratch in Python using NumPy, and then show me the standard way to do it using SciPy. Explain the code line-by-line."*

D. Generating Custom Practice Scenarios
Textbook problems can be dry. Ask AI to generate context-specific problems.
* **Prompt Example:** *"Generate three word problems involving Poisson distributions, but set them in the context of a university library's book checkout system. Provide the solutions hidden under spoiler tags or at the very end."*

⚠️ Pitfalls to Avoid
* **Hallucinated Math:** LLMs are language models, not calculators. They frequently make arithmetic errors. **Always verify the final calculations yourself or use a computational tool (like Wolfram Alpha or Python) to check the math.**

---

4. Subject 2: Database Management Systems (DBMS)
Core Challenges: Mastering SQL syntax, understanding normalization theory, designing efficient schemas, and optimizing query performance.

How to Use GenAI
A. SQL Query Generation, Debugging, and Optimization
AI excels at writing and fixing SQL. Use it to understand *why* a query works or fails.
* **Prompt Example (Debugging):** *"I am getting an 'ORA-00904: invalid identifier' error in this SQL query: [insert query]. Explain what this error means in plain English, identify the exact line causing it, and suggest a fix."*
* **Prompt Example (Optimization):** *"Here is a SQL query that is running slowly on a table with 10 million rows: [insert query]. Suggest three ways to optimize it, including index

recommendations and query rewriting. Explain the execution plan implications of your suggestions."*

B. Schema Design and Normalization
Normalization (1NF, 2NF, 3NF, BCNF) is highly theoretical. Use AI to critique your designs.
*   **Prompt Example:** *"I am designing a database for a university course registration system. Here are my tables and attributes: [insert attributes]. Act as a strict database architect. Review this schema, identify any update anomalies, and guide me through normalizing it to Third Normal Form (3NF)."*

C. Visualizing ER Diagrams
Since text-based AI cannot draw images directly, use it to generate code for diagramming tools.
*   **Prompt Example:** *"Generate Mermaid.js code for an Entity-Relationship (ER) diagram representing an e-commerce database with Customers, Orders, Products, and OrderItems. Include primary keys, foreign keys, and cardinality (1:N, M:N)."* *(You can then paste this code into Mermaid Live Editor to see the visual diagram).*

D. Interview and Exam Prep
*   **Prompt Example:** *"Act as a Senior Database Administrator. Ask me 5 progressively difficult interview questions about database indexing (B-Trees vs. Hash indexes). Wait for my answer to each question before providing feedback and asking the next one."*

⚠️ Pitfalls to Avoid
*   **Dialect Confusion:** AI might mix up SQL dialects (e.g., writing PostgreSQL syntax when you need MySQL). Always specify the dialect in your prompt (e.g., *"Write this in T-SQL for SQL Server"*).

---

5. Subject 3: Software Engineering (SE)
Core Challenges: Moving beyond writing code to designing scalable systems, managing the software development lifecycle (SDLC), applying design patterns, and ensuring code quality.

How to Use GenAI
A. Architecture and Design Patterns
Use AI to understand how high-level concepts apply to your specific projects.
*   **Prompt Example:** *"I am building a food delivery app. Explain how the Microservices architecture would apply to this project compared to a Monolithic architecture. List the pros and cons for this specific use case, and suggest which domain boundaries (services) I should create."*

B. Code Review and Refactoring (SOLID Principles)
Paste your code and ask the AI to critique it based on industry standards.
*   **Prompt Example:** *"Review the following Java class for a User Authentication system. Critique it based on the SOLID principles. Point out any violations of the Single Responsibility Principle or Open/Closed Principle, and provide a refactored version of the code."*

C. Agile Methodologies and Documentation
SE involves heavy documentation and project management. AI can draft these quickly.
*   **Prompt Example:** *"I need to write a Product Requirements Document (PRD) for a new 'Dark Mode' feature in our web app. Generate a template including User Stories, Acceptance Criteria, and Technical Constraints. Fill it out with realistic examples for the Dark Mode feature."*

D. Automated Testing and Edge Cases
AI is excellent at thinking of edge cases you might miss.
*   **Prompt Example:** *"Here is a Python function that calculates the discount for a shopping cart: [insert code]. Act as a QA engineer. List 10 edge cases I should test (e.g., negative prices, empty carts, overlapping promo codes). Then, write the corresponding PyTest unit tests for these edge cases."*

E. Generating UML and Sequence Diagrams
Similar to ER diagrams, use AI to generate PlantUML or Mermaid code for system architecture.
*   **Prompt Example:** *"Generate PlantUML code for a Sequence Diagram showing the interaction between a Client, an API Gateway, an Auth Service, and a Database during a user login process."*

⚠️ Pitfalls to Avoid
*   **Security Vulnerabilities:** AI often writes functional code that is insecure (e.g., susceptible to SQL injection or XSS). Always ask the AI: *"Are there any security vulnerabilities in this code, and how can I fix them?"*
*   **Over-reliance for Architecture:** Do not let AI design your entire system architecture without understanding the trade-offs. You must be able to defend your architectural choices in class and in interviews.

---

6. Advanced Prompting Framework for Students
To get the best results across all subjects, use the **CREATE** framework for your prompts:
*   **C - Context:** Who are you and what is the situation? *(e.g., "I am a junior CS student...")*
*   **R - Request:** What exactly do you want? *(e.g., "Explain the CAP theorem...")*
*   **E - Explanation:** How should it be explained? *(e.g., "...using a restaurant analogy...")*
*   **A - Audience:** Who is the output for? *(e.g., "...for someone who has no business background.")*
*   **T - Tone:** What is the desired tone? *(e.g., "Keep it academic but engaging.")*
*   **E - Extras:** Any constraints? *(e.g., "Keep it under 300 words and use bullet points.")*

---

7. Academic Integrity and Ethical Considerations
Using GenAI in university requires strict adherence to ethical guidelines to avoid plagiarism and academic misconduct.

1. **Know Your University's Policy:** Always check your syllabus and university guidelines. Some professors allow AI for brainstorming; others ban it entirely.
2. **Disclosure:** If in doubt, disclose your use of AI. Add a statement to your assignment: *"Generative AI was used to brainstorm initial ideas and debug SQL syntax. All final analysis, writing, and code logic were produced by the student."*
3. **The "Blank Page" Rule:** Never start an assignment by asking AI to write it. Start with a blank page, do your own thinking, and use AI only to refine, expand, or debug your existing work.
4. **Data Privacy:** Do not upload proprietary university data, unpublished research, or sensitive personal information into public AI models.
5. **Citation:** If you use an AI-generated idea, text, or code snippet, cite it according to your professor's preferred format (e.g., IEEE, APA) or the specific AI citation guidelines provided by your institution.

---

8. Conclusion
Generative AI is the most powerful tutoring tool available to university students today. However, its value is entirely dependent on the student's intent. If used to bypass the learning process, it will result in a superficial understanding and poor performance in technical interviews. If used as an interactive tutor, a rigorous code reviewer, and a conceptual simplifier, it can dramatically accelerate a student's mastery of Probability & Statistics, DBMS, and Software Engineering.

The ultimate goal is not to let AI do the thinking for you, but to use AI to elevate the quality and depth of your own thinking.

3) X.com/i/grok

Generative AI as a Learning Accelerator for University Students in Probability & Statistics, Database Management Systems, and Software EngineeringGenerative AI tools (ChatGPT, Claude, Gemini, Grok, specialized coding assistants such as GitHub Copilot or Cursor, and domain-specific models) can significantly enhance learning when used as interactive tutors, explainer engines, practice generators, and feedback systems rather than answer factories. The goal is deeper understanding, active engagement, and skill transfer—not passive consumption or academic dishonesty.Below is a practical, subject-specific report on effective use.1. Foundational Principles for Productive Use

- Treat AI as a Socratic tutor: Ask for explanations, analogies, step-by-step reasoning, alternative approaches, and critiques of your own work.
- Always verify outputs against textbooks, lecture notes, official documentation, or trusted sources. AI can hallucinate formulas, proofs, SQL syntax, or design patterns.
- Practice retrieval and application: Generate problems, solve them yourself first, then compare.
- Combine with traditional methods: Attend lectures, complete assigned work, form study groups, and use AI for clarification and extension.

- Prompt engineering matters: Be specific (“Explain the Central Limit Theorem with a concrete numerical example using n=30 samples from a skewed distribution and show the sampling distribution of the mean”), provide context (your current knowledge level, course syllabus topics), and iterate.
- Track learning: Keep a notebook of prompts, AI responses, your corrections, and what you still find unclear.

2. Probability and StatisticsGenerative AI excels at conceptual clarification, visualization guidance, derivation walkthroughs, simulation ideas, and personalized problem generation.Core techniques:

- Conceptual explanations and analogies: Request multiple levels (intuitive → formal → mathematical). Example: “Explain Bayes’ theorem first with a medical testing analogy, then derive it formally, then apply it to a spam filter problem.”
- Derivations and proofs: Ask for step-by-step derivations of expected value, variance of linear combinations, maximum likelihood estimators, or properties of common distributions (Binomial, Poisson, Normal, Exponential, Gamma, etc.). Request alternative proofs or common student mistakes.
- Worked examples and counterexamples: Generate numerical examples of the Law of Large Numbers, Central Limit Theorem, hypothesis testing (Type I/II errors, p-values, power), confidence intervals, regression diagnostics, or ANOVA.
- Simulation and computational thinking: Prompt for pseudocode or Python/R code using NumPy, SciPy, or matplotlib to simulate sampling distributions, Monte Carlo methods, bootstrap, or Markov chains. Run the code yourself (or in a sandbox) and interpret results.
- Problem generation and adaptive practice: “Generate 5 medium-difficulty problems on conditional probability involving Bayes’ theorem with solutions hidden. After I attempt them, provide feedback.” Escalate difficulty or focus on weak areas (e.g., continuous random variables, moment-generating functions, multivariate distributions).
- Data analysis workflows: Upload or describe datasets and ask for guidance on exploratory data analysis, choosing appropriate tests, interpreting output, or diagnosing assumptions (normality, independence, homoscedasticity).
- Visualization support: Request descriptions or code for density plots, Q-Q plots, residual plots, or interactive explanations of concepts such as covariance vs. correlation.

Advanced uses: Study design of experiments, Bayesian inference, stochastic processes, or machine learning prerequisites (likelihood, information theory, regularization). Ask AI to connect probability concepts to real research papers or industry applications.Limitations to watch: AI may give incorrect probability calculations for complex dependent events or misstate asymptotic results. Always recompute critical values or run simulations independently.3. Database Management Systems (DBMS)AI is highly effective for SQL mastery, schema design, query optimization understanding, transaction concepts, and NoSQL comparisons.Core techniques:

- Schema design and normalization: Describe a real-world scenario (university course registration, e-commerce, hospital records) and ask for ER diagrams (textual or mermaid syntax), conversion to relational schemas, identification of functional dependencies, and step-by-step normalization to 3NF/BCNF with justification.
- SQL practice: Generate tables with sample data, then request progressive queries (basic SELECT → joins → subqueries → window functions → CTEs → recursive queries). Ask AI to explain execution order, why a query is inefficient, or how indexes would help.
- Query debugging and optimization: Paste your (incorrect or slow) query and ask for diagnosis, corrected version, EXPLAIN plan interpretation, and indexing recommendations.
- Conceptual depth: Explanations of ACID properties with concrete failure scenarios, concurrency control (locking, MVCC, deadlocks), recovery (WAL, checkpoints), indexing structures (B+ trees, hash indexes), and query processing (parsing, optimization, join algorithms).
- Hands-on projects: Design a full mini-database for a domain, generate DDL, sample DML, views, stored procedures/triggers, and integrity constraints. Request test cases for referential integrity or isolation levels.
- Beyond relational: Compare SQL vs. document, key-value, graph, or column-family stores. Ask for MongoDB aggregation pipelines, Neo4j Cypher examples, or when to choose which model.
- Security and administration: Guidance on roles/permissions, SQL injection prevention (parameterized queries), backup strategies, and performance monitoring concepts.

Practical workflow: Use AI alongside actual DBMS tools (PostgreSQL, MySQL, SQLite, or cloud free tiers). Write queries yourself, execute them, then refine with AI feedback. Request migration scripts or data modeling critiques.Limitations: AI-generated schemas may miss real-world constraints or scalability considerations. Always test queries on real engines; syntax and optimizer behavior differ across systems.4. Software EngineeringAI shines as a pair-programmer, design critic, documentation generator, testing assistant, and process explainer.Core techniques:

- Requirements and design: Convert vague problem statements into user stories, use cases, UML diagrams (class, sequence, activity—described in text or mermaid), architecture sketches (layered, microservices, event-driven), and design pattern applications (Factory, Observer, Strategy, Repository, etc.) with trade-offs.
- Coding and implementation: Generate boilerplate, refactor existing code, explain idiomatic usage in languages (Java, Python, C++, JavaScript/TypeScript), or implement algorithms/data structures with complexity analysis. Use tools that integrate with IDEs for real-time suggestions.
- Testing and quality: Generate unit tests (JUnit, pytest), integration test scenarios, edge cases, and property-based testing ideas. Ask for code reviews focusing on readability, maintainability, security, and SOLID

principles. Request mutation testing concepts or coverage improvement suggestions.

- Version control, CI/CD, and processes: Explanations of Git workflows (branching strategies, rebase vs. merge), writing good commit messages, setting up simple CI pipelines, agile ceremonies, estimation techniques, and technical debt management.
- Documentation and communication: Generate API docs, README files, architecture decision records (ADRs), and clear explanations of complex modules for teammates or future self.
- Debugging and troubleshooting: Paste stack traces or buggy code and request systematic diagnosis (root cause analysis, logging strategies, common pitfalls).
- Project-scale learning: Build progressively larger projects (e.g., a simple web app → add authentication, database layer, testing, deployment). At each stage, ask AI for architectural advice, alternative implementations, and critiques of your choices.
- Soft and professional skills: Practice writing design rationales, code review comments, or responses to requirements changes.

Best integration: Use AI inside development environments. Write the core logic yourself, use AI for acceleration and review, then understand every line generated. Study clean code principles and design patterns independently so you can evaluate AI suggestions critically.Limitations: AI code can contain subtle bugs, security issues (e.g., insecure defaults), or outdated practices. It may over-engineer or under-engineer solutions. Never submit AI-generated code as your own without understanding and significant modification; most universities require disclosure or prohibit unacknowledged use.5. Cross-Cutting Best Practices and Risk Management

- Active learning loop: Prompt → Attempt solution yourself → Compare/ critique AI output → Refine understanding → Apply to new problems → Teach back (explain to AI or peers).
- Personalization: Share your syllabus topics, exam style, or weak areas. Request spaced-repetition style quizzes or cumulative reviews.
- Multimodal support: Ask for analogies, visual descriptions, code + diagrams, or connections across subjects (e.g., statistical concepts in A/B testing for software features, probability in randomized algorithms, database design informed by data modeling for ML).
- Collaboration and ethics: Use AI for brainstorming study plans or group project scaffolding, but complete individual assessments independently. Follow your university's academic integrity policy strictly—most treat undisclosed AI use as plagiarism.
- Complementary tools: Pair with official documentation, textbooks (e.g., Ross for probability, Silberschatz/Elmasri for DBMS, Pressman or Sommerville for software engineering), online judges, local IDEs/DBMSs, and version control.
- Limitations and pitfalls: Hallucinations, over-reliance reducing critical thinking, surface-level understanding, and potential skill atrophy in debugging or derivation. Regularly solve problems without AI to measure true competence.

- Measuring effectiveness: Track exam performance, ability to solve novel problems, speed of grasping new topics, and confidence in explaining concepts without notes.

6. Sample Weekly Workflow
    1. After lecture: Summarize key points yourself, then ask AI to expand, correct misconceptions, or provide additional examples.
    2. Homework phase: Attempt problems first; use AI for hints, alternative solutions, or verification.
    3. Project work: Use AI for design feedback, code acceleration, testing, and documentation while owning the architecture and logic.
    4. Exam preparation: Generate practice sets, timed mock exams, and concept maps; review weak areas deeply.
    5. Reflection: Weekly review of what AI helped versus where independent effort was essential.

ConclusionWhen used deliberately as an always-available, infinitely patient tutor and critique partner, generative AI can accelerate mastery of probability & statistics (conceptual depth + computational intuition), database systems (design + query fluency), and software engineering (design thinking + implementation quality). The students who benefit most treat AI as a force multiplier for their own effort rather than a replacement for it. Combine rigorous self-practice, verification against authoritative sources, and ethical use, and these tools become one of the most powerful learning advantages available to university students today.Focus on understanding the “why” behind every explanation, formula, query, or design decision the AI provides. That is the path from temporary assistance to lasting competence.

Grok (DeepSearch) — A clean, well-organized practical guide with principles up front and concrete example prompts per subject. Good as a how-to reference, but fairly generic and light on evidence.

DeepSeek (R1) — The most rigorous and evidence-backed, citing real studies and numbers (e.g. test scores jumping from 39% to 70.5% with AI tutoring, 82% SQL improvement vs. 64% debugging-dependency risk). It also flagged a sharper insight: Copilot can make you faster without improving actual understanding unless you verify the code yourself.

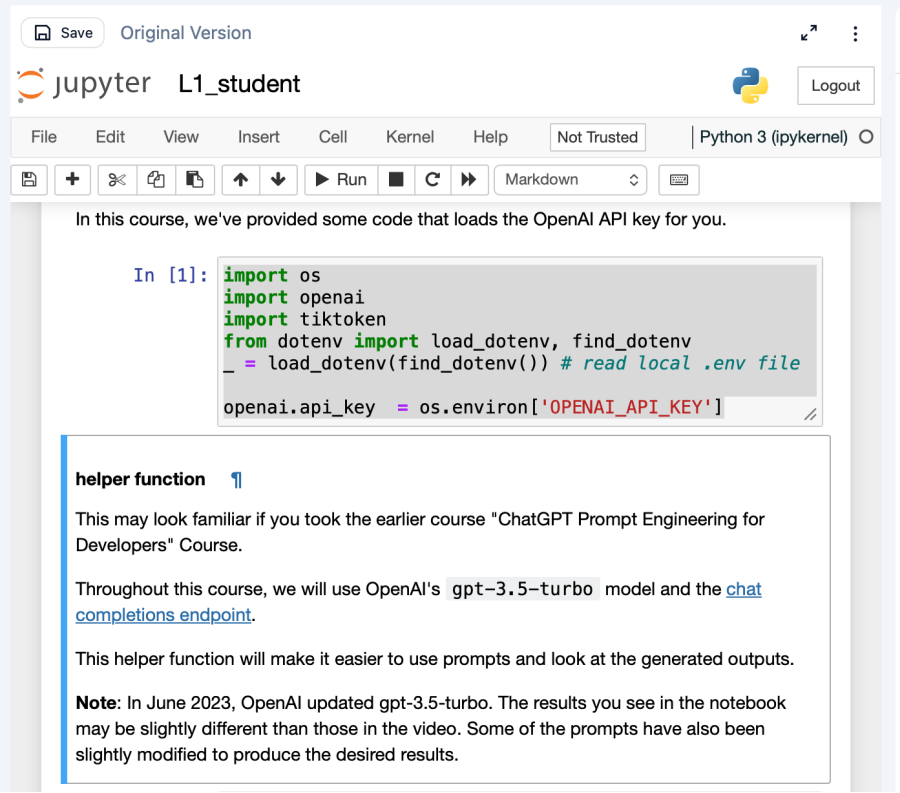
Qwen (Thinking) — The most practical toolkit, built around a reusable prompting framework (CREATE) with copy-paste prompt templates, including diagram-generation prompts (Mermaid/PlantUML) the other two didn't emphasize. Weakest on citing research to support its claims.

Reflection

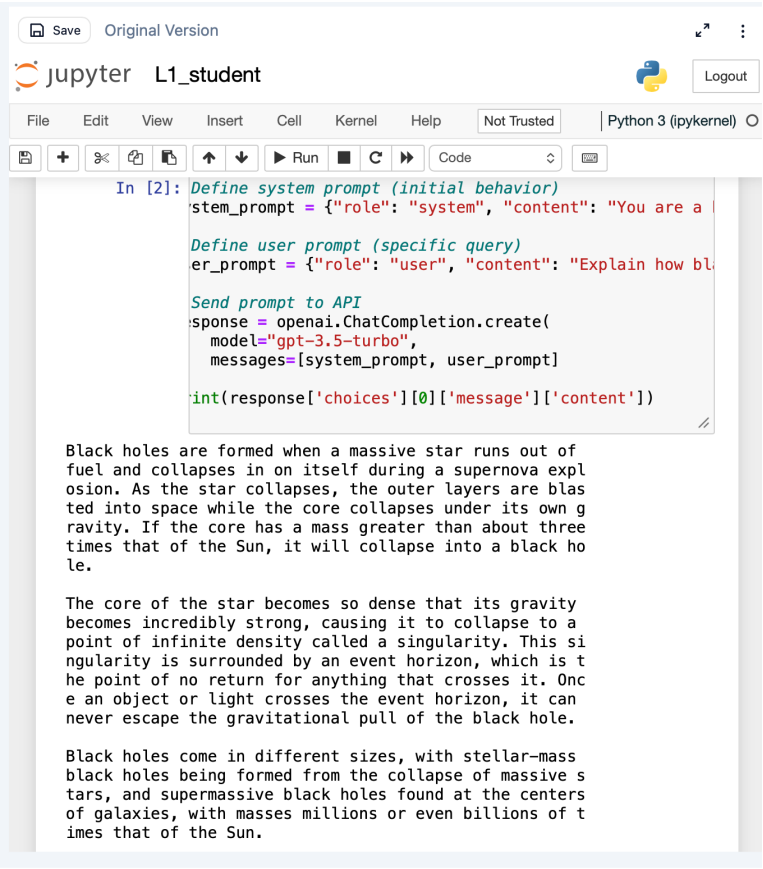
For this exercise, I used Grok, DeepSeek R1, and Qwen (with their reasoning/deep-search modes enabled) to generate a report on using generative AI for learning probability & statistics, DBMS, and software engineering. Grok gave a balanced, textbook-style overview but was fairly generic. Qwen was the most immediately practical, offering a reusable prompting framework (CREATE) and ready-made prompt templates, including ones for generating ER and sequence diagrams. DeepSeek R1 stood out as the most rigorous — it cited real studies with concrete numbers (e.g. test scores rising from 39% to 70.5% with AI-assisted tutoring) and raised an important caution: tools like Copilot can make you faster without actually improving understanding, unless you deliberately verify what the AI produces. In my opinion, DeepSeek R1 produced the best report overall, since it was the most evidence-based and honest about the risks of over-reliance. Going forward, I'd combine Qwen's practical prompt templates with DeepSeek's emphasis on verifying my own understanding rather than accepting AI output at face value.

Exercise 4

4)



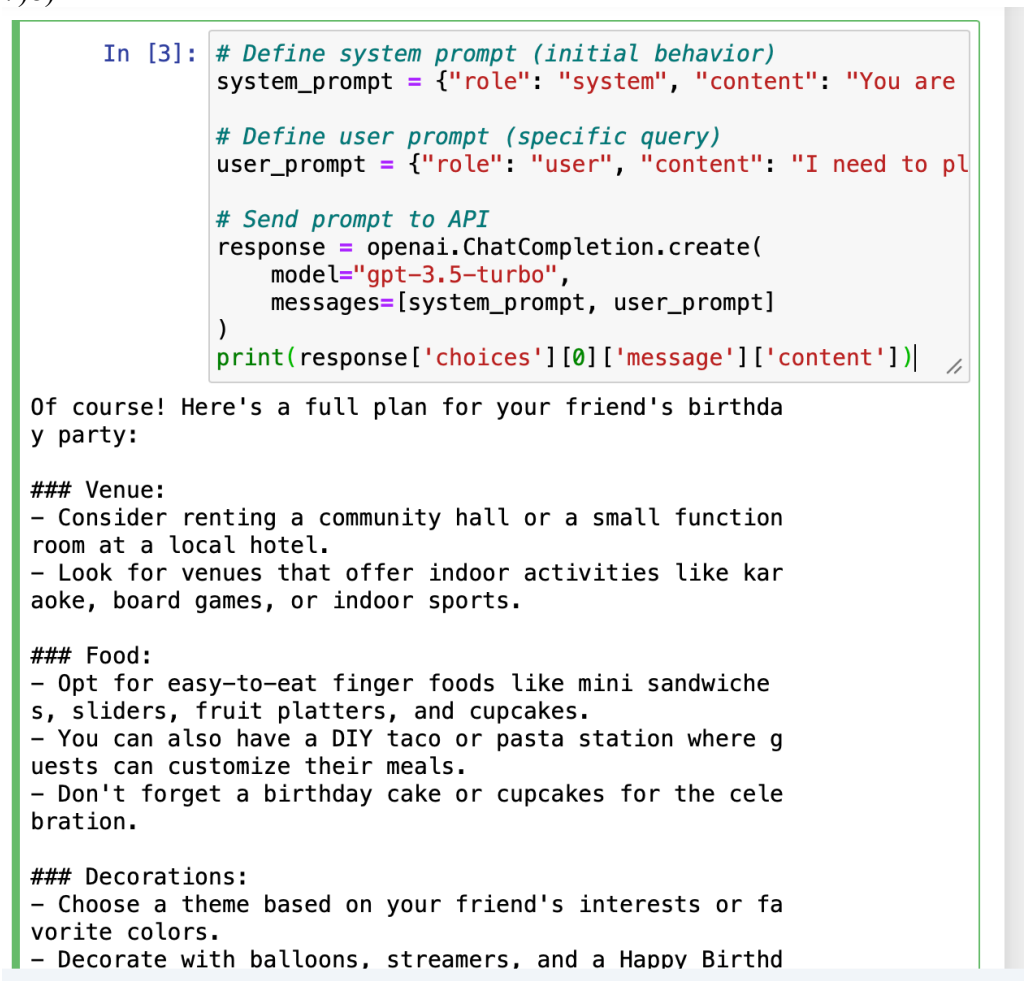
5)



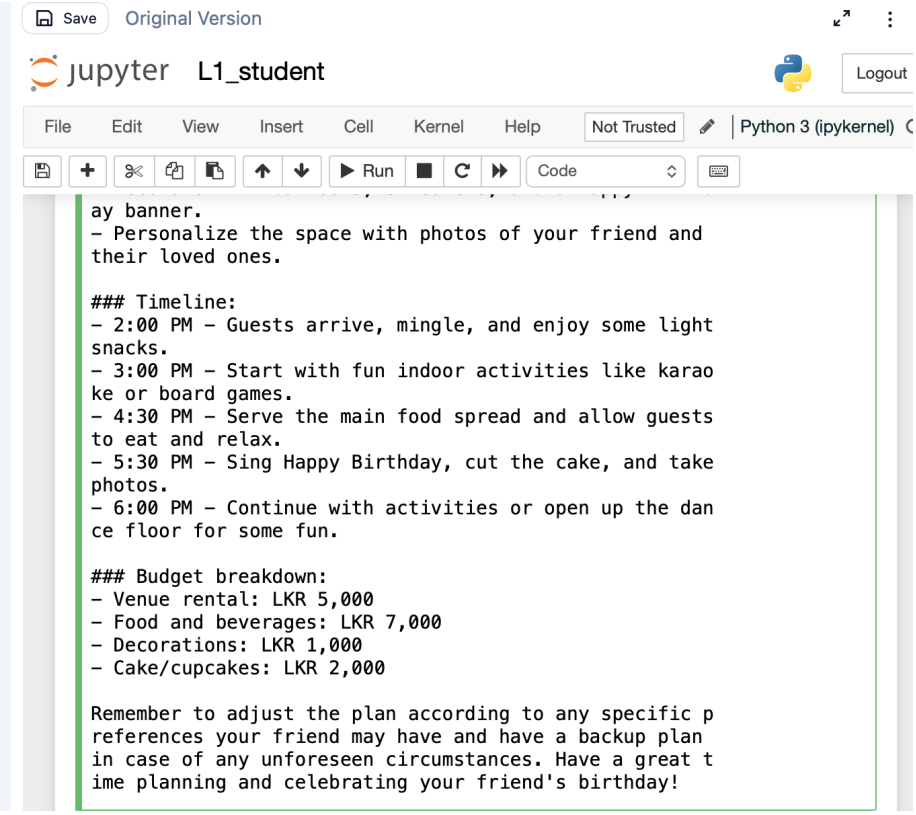
6)

```
# Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a helpful assistant who is an expert event planner, skilled at organizing budgets, venues, timelines, and activities for events of all sizes."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "I need to plan my friend's birthday party. We have around 15 guests, a budget of LKR 15,000, and want an indoor venue with some fun activities. Can you suggest a full plan including venue ideas, food, decorations, and a rough timeline?"}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

7)8)

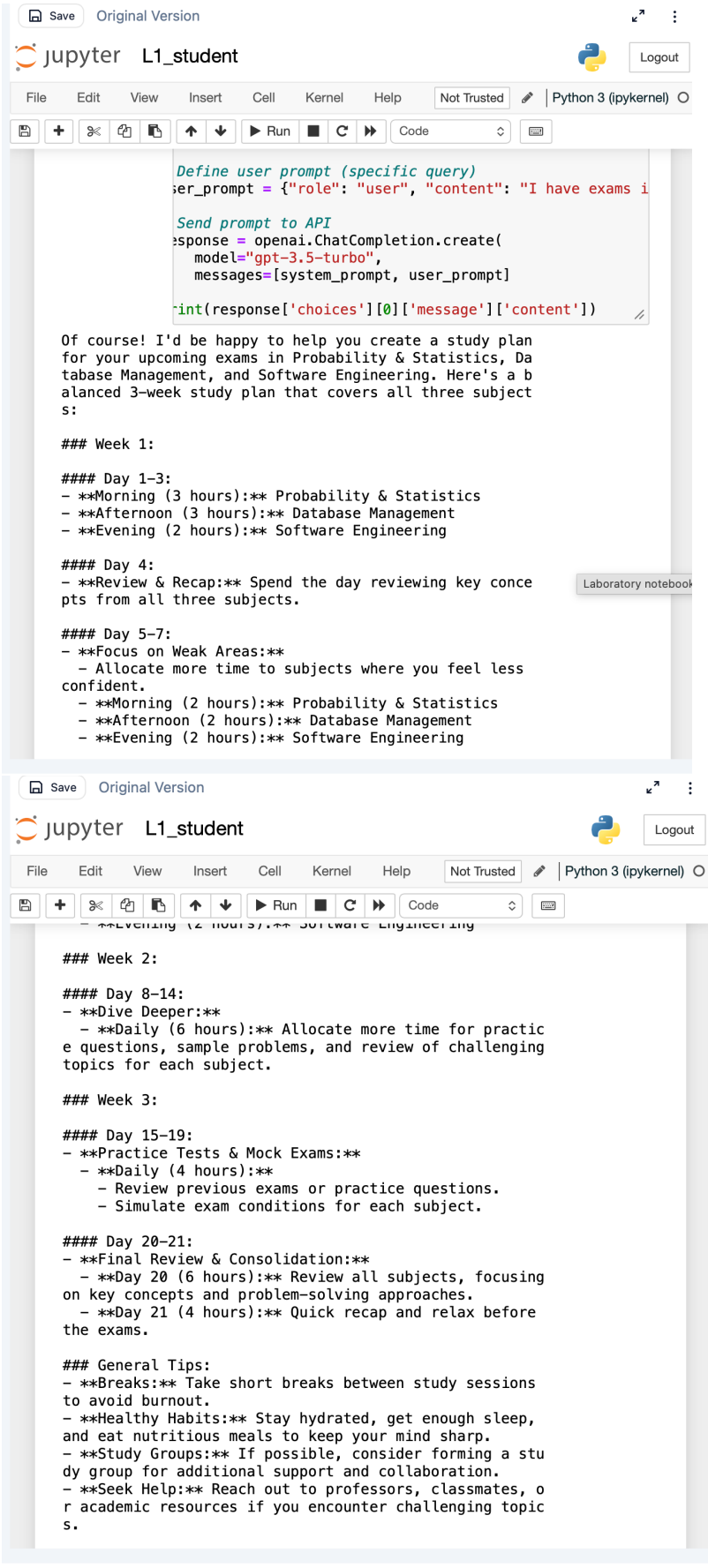


```
In [3]: # Define system prompt (initial behavior)
        system_prompt = {"role": "system", "content": "You are

        # Define user prompt (specific query)
        user_prompt = {"role": "user", "content": "I need to pl

        # Send prompt to API
        response = openai.ChatCompletion.create(
            model="gpt-3.5-turbo",
            messages=[system_prompt, user_prompt]
        )
        print(response['choices'][0]['message']['content'])

Of course! Here's a full plan for your friend's birthda
y party:

### Venue:
- Consider renting a community hall or a small function
room at a local hotel.
- Look for venues that offer indoor activities like kar
aoke, board games, or indoor sports.

### Food:
- Opt for easy-to-eat finger foods like mini sandwiche
s, sliders, fruit platters, and cupcakes.
- You can also have a DIY taco or pasta station where g
uests can customize their meals.
- Don't forget a birthday cake or cupcakes for the cele
bration.

### Decorations:
- Choose a theme based on your friend's interests or fa
vorite colors.
- Decorate with balloons, streamers, and a Happy Birthd
```



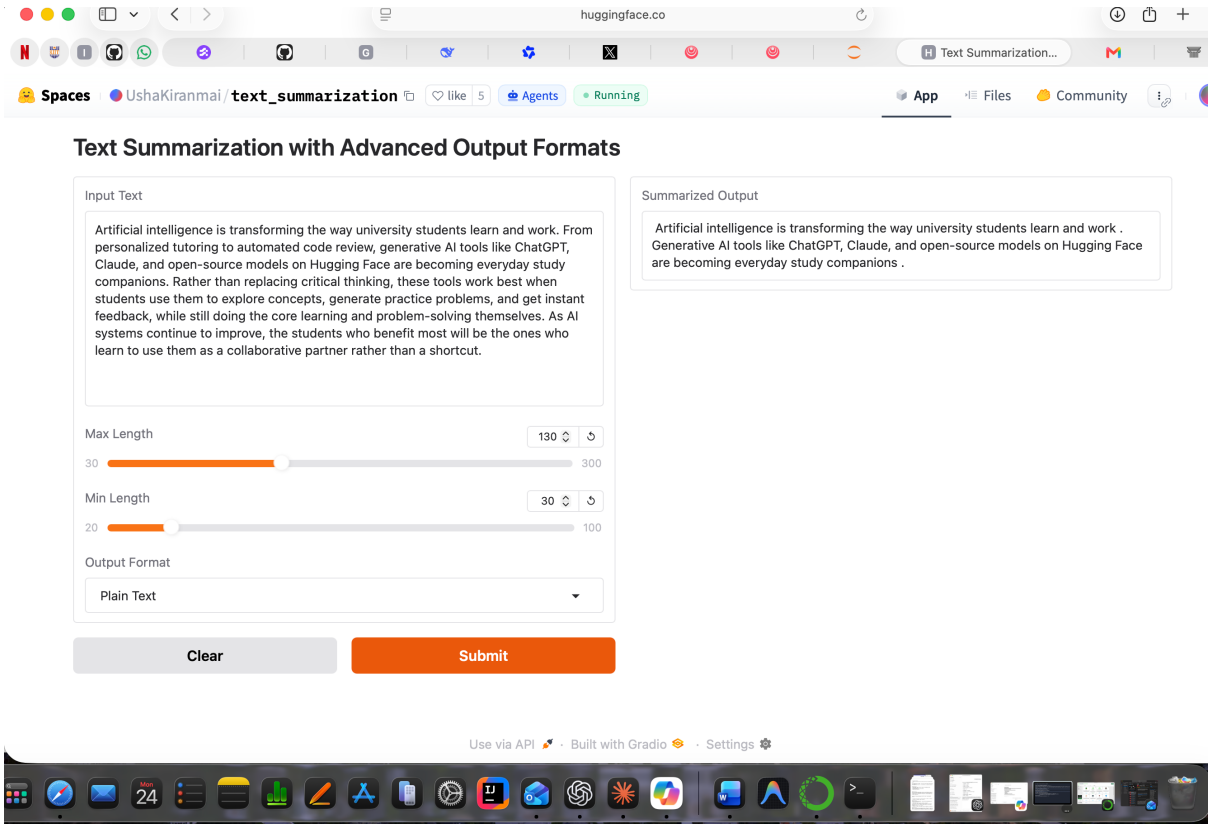
9)

```
# Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a helpful assistant who is an expert academic study coach, specializing in helping university students build effective study schedules and revision strategies."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "I have exams in Probability & Statistics, Database Management, and Software Engineering in three weeks. Can you help me create a balanced 3-week study plan that covers all three subjects?"}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```


Save
Original Version
jupyter
L1_student
Logout
File
Edit
View
Insert
Cell
Kernel
Help
Not Trusted
Python 3 (ipykernel)
Run
Code
Define user prompt (specific query)
ser_prompt = {"role": "user", "content": "I have exams i
Send prompt to API
esponse = openai.ChatCompletion.create(
model="gpt-3.5-turbo",
messages=[system_prompt, user_prompt]
rint(response['choices'][0]['message']['content'])
Of course! I'd be happy to help you create a study plan
for your upcoming exams in Probability & Statistics, Da
tabase Management, and Software Engineering. Here's a b
alanced 3-week study plan that covers all three subject
s:
Week 1:
Day 1-3:
- **Morning (3 hours):** Probability & Statistics
- **Afternoon (3 hours):** Database Management
- **Evening (2 hours):** Software Engineering
Day 4:
- **Review & Recap:** Spend the day reviewing key conce
pts from all three subjects.
Day 5-7:
- **Focus on Weak Areas:**
- Allocate more time to subjects where you feel less
confident.
- **Morning (2 hours):** Probability & Statistics
- **Afternoon (2 hours):** Database Management
- **Evening (2 hours):** Software Engineering
Laboratory notebook
Save
Original Version
jupyter
L1_student
Logout
File
Edit
View
Insert
Cell
Kernel
Help
Not Trusted
Python 3 (ipykernel)
Run
Code
Week 2:
Day 8-14:
- **Dive Deeper:**
- **Daily (6 hours):** Allocate more time for practic
e questions, sample problems, and review of challenging
topics for each subject.
Week 3:
Day 15-19:
- **Practice Tests & Mock Exams:**
- **Daily (4 hours):**
- Review previous exams or practice questions.
- Simulate exam conditions for each subject.
Day 20-21:
- **Final Review & Consolidation:**
- **Day 20 (6 hours):** Review all subjects, focusing
on key concepts and problem-solving approaches.
- **Day 21 (4 hours):** Quick recap and relax before
the exams.
General Tips:
- **Breaks:** Take short breaks between study sessions
to avoid burnout.
- **Healthy Habits:** Stay hydrated, get enough sleep,
and eat nutritious meals to keep your mind sharp.
- **Study Groups:** If possible, consider forming a stu
dy group for additional support and collaboration.
- **Seek Help:** Reach out to professors, classmates, o
r academic resources if you encounter challenging topic
s.

Exercise 5

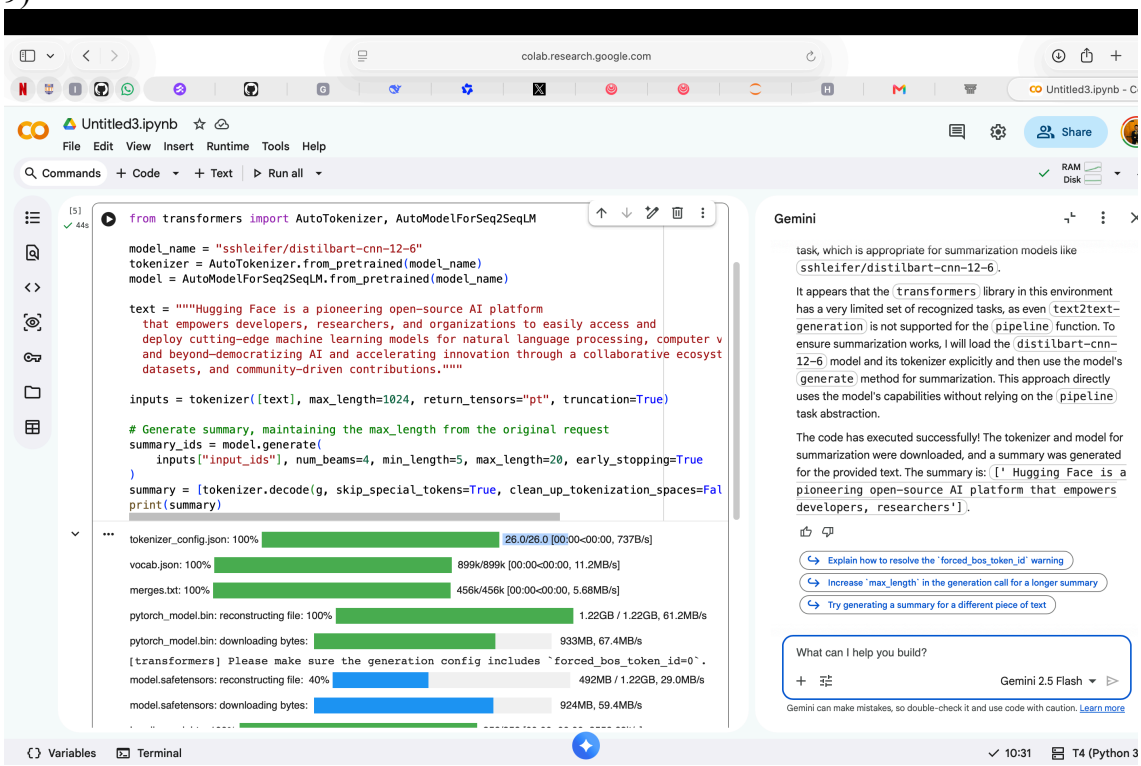
3)4)



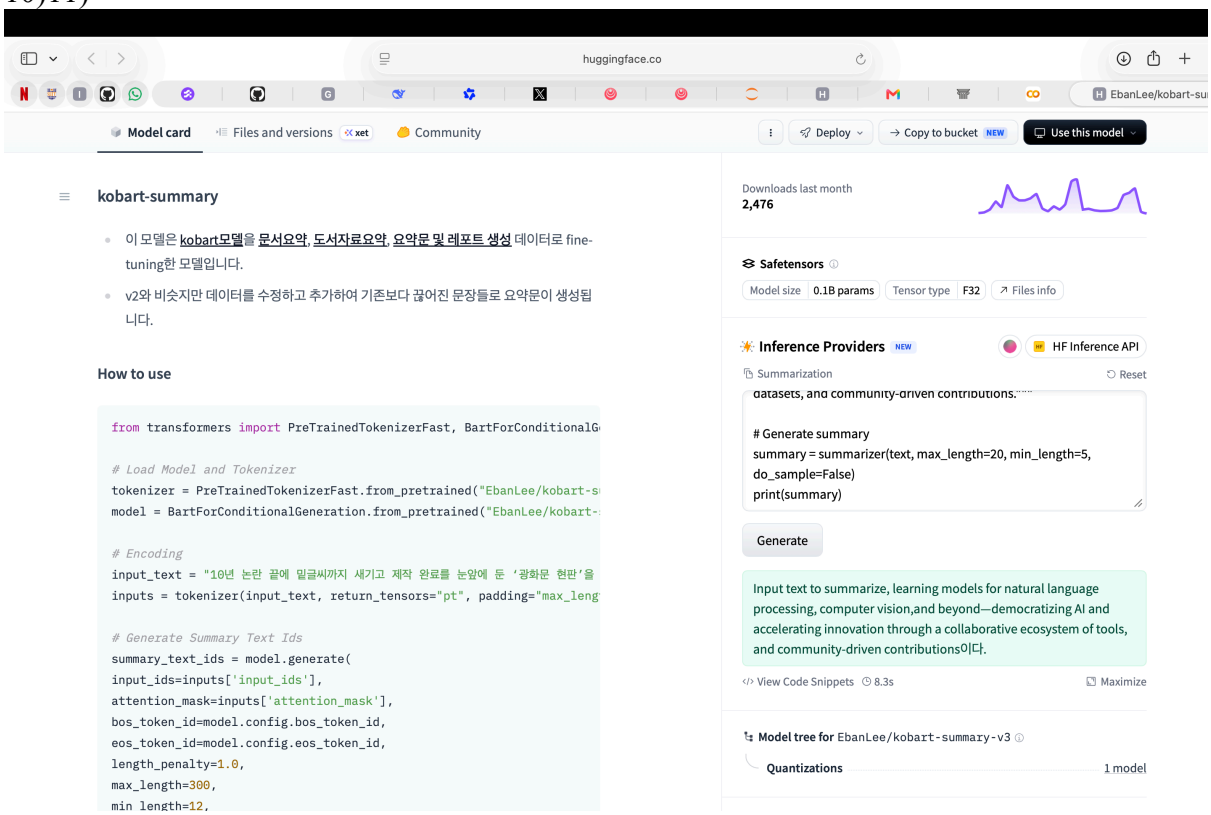
5)

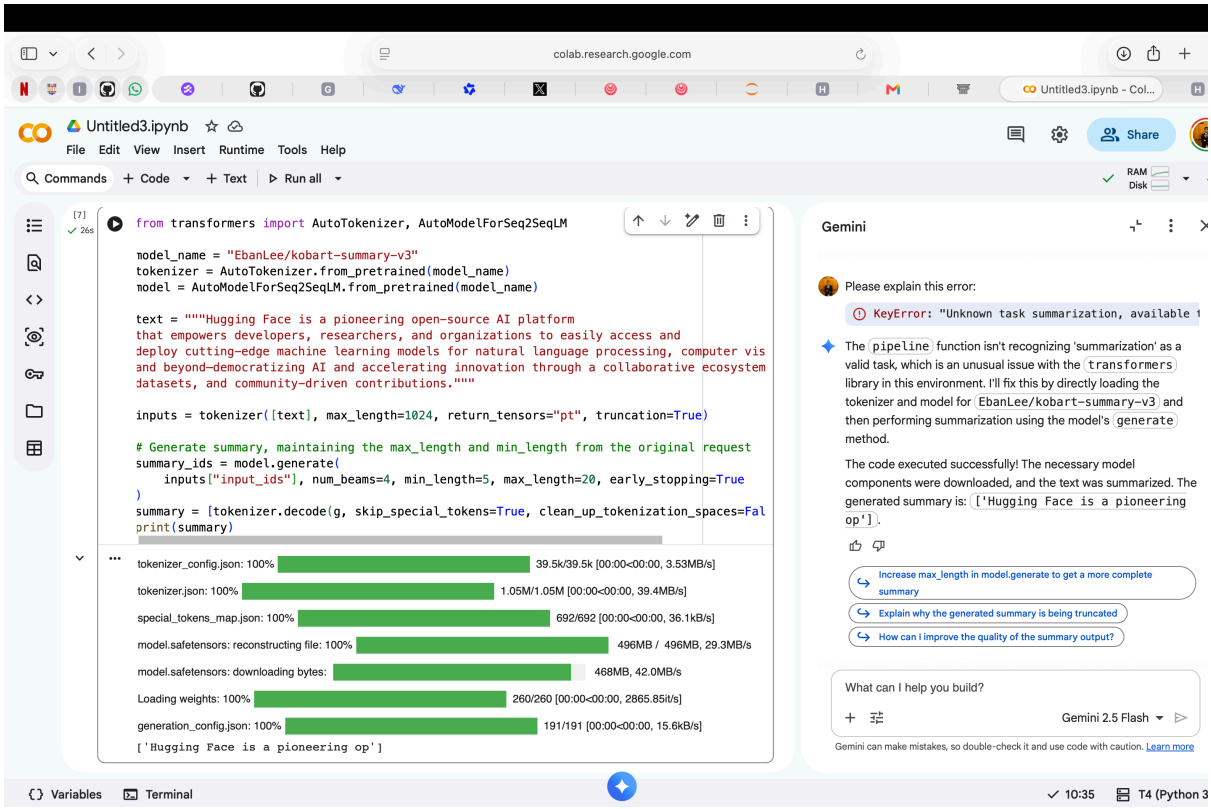
- **Publisher:** Qwen (Alibaba's Qwen team)
- **Task type:** Image-Text-to-Text — meaning it's a multimodal model that can take both images and text as input and generate text output (not pure text-only)
- **Popularity:** 12.3k likes, 99.7k followers on the Qwen org — a very widely used model
- **Frameworks:** Built on Transformers, uses Safetensors format (a safer/faster alternative to the older `.bin` checkpoint format)
- **Architecture family:** Tagged `qwen3_5`, suggesting it's part of the Qwen3.5 architecture line
- **Conversational:** Tagged for chat/conversational use, so it's designed to work in a dialogue format, not just single-shot completions
- **License:** Apache 2.0 — a permissive open-source license, meaning it can be used, modified, and even used commercially with minimal restriction
- **Has Eval Results:** meaning benchmark scores are published on its model card if you want to dig into performance numbers

9)

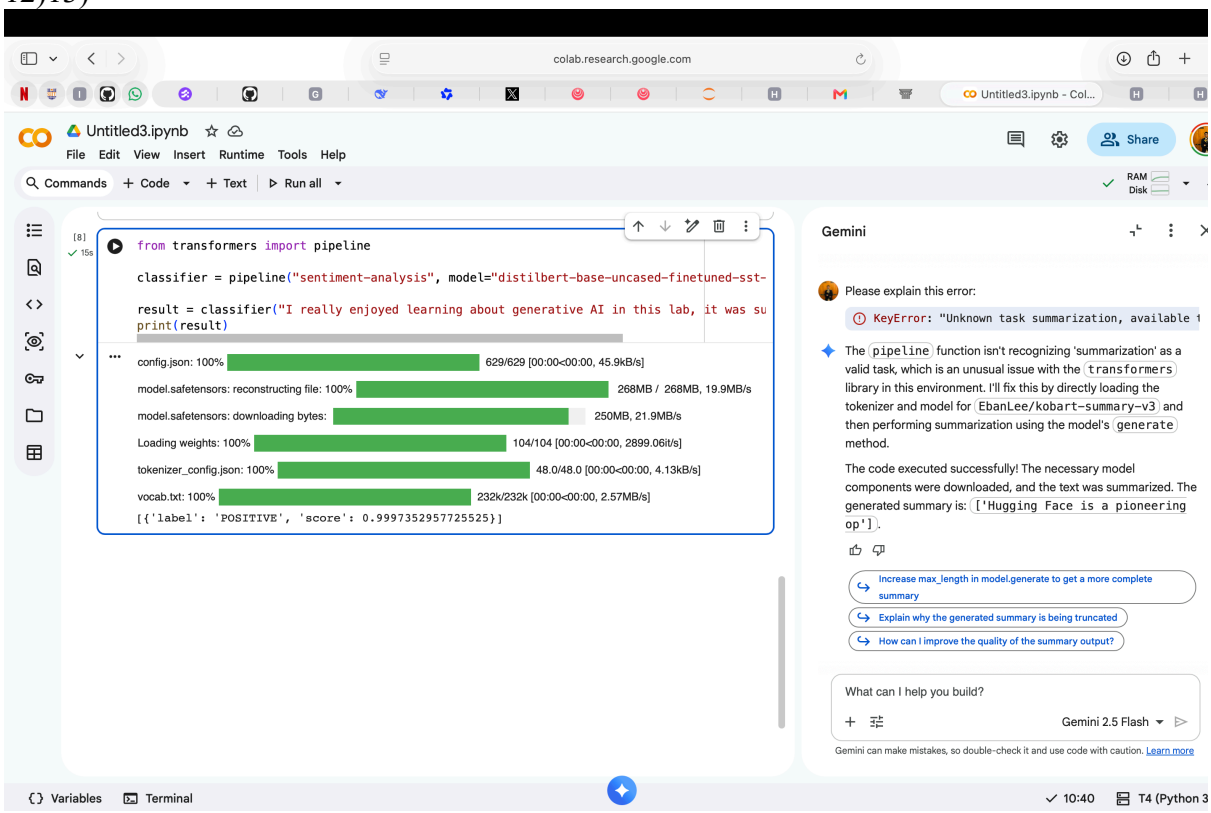

colab.research.google.com
Untitled3.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
model_name = "sshleifer/distilbart-cnn-12-6"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
text = """Hugging Face is a pioneering open-source AI platform
that empowers developers, researchers, and organizations to easily access and
deploy cutting-edge machine learning models for natural language processing, computer v
and beyond—democratizing AI and accelerating innovation through a collaborative ecosyst
datasets, and community-driven contributions."""
inputs = tokenizer([text], max_length=1024, return_tensors="pt", truncation=True)
# Generate summary, maintaining the max_length from the original request
summary_ids = model.generate(
inputs["input_ids"], num_beams=4, min_length=5, max_length=20, early_stopping=True
)
summary = [tokenizer.decode(g, skip_special_tokens=True, clean_up_tokenization_spaces=Fal
print(summary)
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 737B/s]
vocab.json: 100% 899k/899k [00:00<00:00, 11.2MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 5.68MB/s]
pytorch_model.bin: reconstructing file: 100% 1.22GB / 1.22GB, 61.2MB/s
pytorch_model.bin: downloading bytes: 933MB, 67.4MB/s
[transformers] Please make sure the generation config includes `forced_bos_token_id=0`.
model.safetensors: reconstructing file: 40% 492MB / 1.22GB, 29.0MB/s
model.safetensors: downloading bytes: 924MB, 59.4MB/s
Gemini
task, which is appropriate for summarization models like sshleifer/distilbart-cnn-12-6.
It appears that the transformers library in this environment has a very limited set of recognized tasks, as even text2text-generation is not supported for the pipeline function. To ensure summarization works, I will load the distilbart-cnn-12-6 model and its tokenizer explicitly and then use the model's generate method for summarization. This approach directly uses the model's capabilities without relying on the pipeline task abstraction.
The code has executed successfully! The tokenizer and model for summarization were downloaded, and a summary was generated for the provided text. The summary is: [' Hugging Face is a pioneering open-source AI platform that empowers developers, researchers'].
Explain how to resolve the `forced_bos_token_id` warning
Increase `max_length` in the generation call for a longer summary
Try generating a summary for a different piece of text
What can I help you build?
Gemini 2.5 Flash
Gemini can make mistakes, so double-check it and use code with caution. Learn more
Variables Terminal
10:31 T4 (Python 3

10)11)


huggingface.co
EbanLee/kobart-su
Model card Files and versions xet Community
Deploy Copy to bucket NEW Use this model
kobart-summary
이 모델은 kobart모델을 문서요약, 도서자료요약, 요약문 및 레포트 생성 데이터로 fine-tuning한 모델입니다.
v2와 비슷지만 데이터를 수정하고 추가하여 기존보다 끊어진 문장들로 요약문이 생성됩니다.
How to use
from transformers import PreTrainedTokenizerFast, BartForConditionalG
# Load Model and Tokenizer
tokenizer = PreTrainedTokenizerFast.from_pretrained("EbanLee/kobart-s
model = BartForConditionalGeneration.from_pretrained("EbanLee/kobart-
# Encoding
input_text = "10년 논란 끝에 밀글씨까지 새기고 제작 완료를 눈앞에 둔 '광화문 현판'을
inputs = tokenizer(input_text, return_tensors="pt", padding="max_leng
# Generate Summary Text Ids
summary_text_ids = model.generate(
input_ids=inputs['input_ids'],
attention_mask=inputs['attention_mask'],
bos_token_id=model.config.bos_token_id,
eos_token_id=model.config.eos_token_id,
length_penalty=1.0,
max_length=300,
min_length=12,
Downloads last month
2,476
Safetensors
Model size 0.1B params Tensor type F32 Files info
Inference Providers NEW HF Inference API
Summarization Reset
datasets, and community-driven contributions."""
# Generate summary
summary = summarizer(text, max_length=20, min_length=5, do_sample=False)
print(summary)
Generate
Input text to summarize, learning models for natural language processing, computer vision,and beyond—democratizing AI and accelerating innovation through a collaborative ecosystem of tools, and community-driven contributions이다.
View Code Snippets 8.3s Maximize
Model tree for EbanLee/kobart-summary-v3
Quantizations 1 model


colab.research.google.com
Untitled3.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
[7] 26s
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
model_name = "EbanLee/kobart-summary-v3"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
text = """Hugging Face is a pioneering open-source AI platform
that empowers developers, researchers, and organizations to easily access and
deploy cutting-edge machine learning models for natural language processing, computer vis
and beyond—democratizing AI and accelerating innovation through a collaborative ecosystem
datasets, and community-driven contributions."""
inputs = tokenizer([text], max_length=1024, return_tensors="pt", truncation=True)
# Generate summary, maintaining the max_length and min_length from the original request
summary_ids = model.generate(
inputs["input_ids"], num_beams=4, min_length=5, max_length=20, early_stopping=True
)
summary = [tokenizer.decode(g, skip_special_tokens=True, clean_up_tokenization_spaces=Fal
print(summary)
tokenizer_config.json: 100% 39.5k/39.5k [00:00<00:00, 3.53MB/s]
tokenizer.json: 100% 1.05M/1.05M [00:00<00:00, 39.4MB/s]
special_tokens_map.json: 100% 692/692 [00:00<00:00, 36.1kB/s]
model.safetensors: reconstructing file: 100% 496MB / 496MB, 29.3MB/s
model.safetensors: downloading bytes: 468MB, 42.0MB/s
Loading weights: 100% 260/260 [00:00<00:00, 2865.85it/s]
generation_config.json: 100% 191/191 [00:00<00:00, 15.6kB/s]
['Hugging Face is a pioneering op']
Gemini
Please explain this error:
KeyError: "Unknown task summarization, available
The pipeline function isn't recognizing 'summarization' as a valid task, which is an unusual issue with the transformers library in this environment. I'll fix this by directly loading the tokenizer and model for EbanLee/kobart-summary-v3 and then performing summarization using the model's generate method.
The code executed successfully! The necessary model components were downloaded, and the text was summarized. The generated summary is: ['Hugging Face is a pioneering op'].
Increase max_length in model.generate to get a more complete summary
Explain why the generated summary is being truncated
How can I improve the quality of the summary output?
What can I help you build?
Gemini 2.5 Flash
Gemini can make mistakes, so double-check it and use code with caution. Learn more
Variables Terminal
10:35 T4 (Python 3

12)13)


colab.research.google.com
Untitled3.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
[8] 15s
from transformers import pipeline
classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-
result = classifier("I really enjoyed learning about generative AI in this lab, it was su
print(result)
config.json: 100% 629/629 [00:00<00:00, 45.9kB/s]
model.safetensors: reconstructing file: 100% 268MB / 268MB, 19.9MB/s
model.safetensors: downloading bytes: 250MB, 21.9MB/s
Loading weights: 100% 104/104 [00:00<00:00, 2899.06it/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.13kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 2.57MB/s]
[{'label': 'POSITIVE', 'score': 0.9997352957725525}]
Gemini
Please explain this error:
KeyError: "Unknown task summarization, available
The pipeline function isn't recognizing 'summarization' as a valid task, which is an unusual issue with the transformers library in this environment. I'll fix this by directly loading the tokenizer and model for EbanLee/kobart-summary-v3 and then performing summarization using the model's generate method.
The code executed successfully! The necessary model components were downloaded, and the text was summarized. The generated summary is: ['Hugging Face is a pioneering op'].
Increase max_length in model.generate to get a more complete summary
Explain why the generated summary is being truncated
How can I improve the quality of the summary output?
What can I help you build?
Gemini 2.5 Flash
Gemini can make mistakes, so double-check it and use code with caution. Learn more
Variables Terminal
10:40 T4 (Python 3